

# Demonstrating BriskState: A Cache-Aware State Backend for Apache Flink

Anonymous Authors  
Anonymous Submission  
Anonymous, Anonymous

## Abstract

Stateful stream processing often becomes memory-bound before it becomes compute-bound. Frequent state lookups, window maintenance, and timer firing amplify cache misses, long object-indirection chains, and heap-management overhead, yet these effects are hard to observe in a live system. This paper presents a demonstration of BriskState, an Apache Flink state backend together with its native execution engine and the existing WordCount and NexMark benchmark artifacts in the repository. The goal is to expose the backend design itself: compact state organization, locality-aware access paths, and their impact on throughput, latency, checkpoint behavior, and state-access hot spots during live runs and trace replay. A lightweight interface is used only to make these behaviors observable and to support side-by-side comparison against heap-based baselines. The paper focuses on how the BriskState design appears in execution traces and how locality-aware storage changes runtime symptoms in state-intensive stream-processing workloads.

## Keywords

stream processing, state backend, BriskState, cache-aware data layout, Apache Flink

## 1 Introduction

Large-state stream-processing pipelines rely on the state backend on nearly every record. In Apache Flink, this path is exercised by point updates, window aggregation, joins, and timer callbacks, so backend behavior can dominate end-to-end performance even when user logic is lightweight [7]. In practice, the bottleneck is frequently the memory hierarchy rather than arithmetic throughput: state accesses trigger pointer chasing, touch multiple cache lines, and create runtime jitter through heap-management activity.

Conventional benchmark plots show final throughput or completion time, yet they do not show why one backend stalls, which namespaces become hot, or how checkpoint traffic and key-group locality change over time. The missing link is the execution evidence needed to connect backend structure with observed runtime behavior.

BriskState addresses this gap with a cache-aware backend integrated with the delivered benchmark harness. The backend follows the implemented Java-control-layer, JNI-bridge, and native-engine split described in the project documentation, while a lightweight interface exposes the resulting behavior through a control panel, topology and state inspector, and time-series views for throughput, latency, and

checkpoint activity. The workflow covers both pure state-stress workloads and realistic streaming jobs within a single experimental path.

BriskState is positioned against three nearby lines of work. Stateful dataflow systems such as MillWheel, Flink, Naiad, Structured Streaming, and the Dataflow model define high-level execution semantics, but the physical organization of state remains largely hidden behind operator APIs and runtime services [2, 3, 5, 7, 10]. In Flink deployments, backend choices usually reduce to in-memory heap structures or embedded persistent stores such as RocksDB, while work on consistent state management and migration, including Flink state management and Megaphone, emphasizes correctness and continuity rather than layout-aware observability [6, 7, 9, 11]. High-performance key-value engines and compact hash-table designs, including FASTER and Swiss-table-style indexing, show that memory layout and probing discipline materially change access cost, and benchmark suites such as NexMark capture the resulting end-to-end outcomes without exposing the runtime state structure behind them [1, 4, 8]. BriskState targets that gap by making a Flink-compatible state backend observable as a running systems component rather than evaluating it only through final aggregate scores.

The paper makes three contributions:

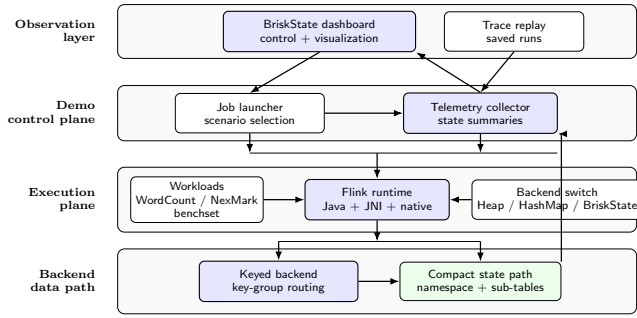
- It presents BriskState together with a runnable workflow for comparing it against standard heap-based baselines under the same Flink workloads.
- It exposes otherwise hidden structures, including key-group routing, namespace occupancy, and checkpoint phases, so locality effects become understandable during execution.
- It packages the current BriskState prototype, benchmark harness, and observation workflow into a reproducible demonstration based on standard Linux deployment.

## 2 BriskState

### 2.1 Overview

Figure 1 summarizes the demonstration architecture. The control plane accepts workload and backend selections from the web interface, launches the requested Flink job, and subscribes to runtime metrics emitted by the benchmark harness, Flink runtime, and lightweight collectors. The data plane runs the target job with either a heap-based baseline or BriskState, then streams summarized metrics and structural summaries back to the interface.

The emphasis is on the backend itself rather than the surrounding interface. Instead of treating the backend as a black



**Figure 1: Architecture of the demonstration.** A web interface is layered on top of the backend and benchmark artifacts, then receives performance telemetry and state summaries from the running job.

box, the interface exposes how state is physically organized and how that organization changes runtime symptoms. In particular, the workflow highlights three views: a job-level comparison panel for backend selection, a state-structure view centered on key-groups and namespaces, and a timeline view that correlates throughput changes with latency spikes and checkpoint phases. The underlying storage path follows compact, cache-conscious hash-table ideas inspired by modern Swiss-table designs [1], while the discussion remains at the systems level.

That systems view maps cleanly to the implemented storage hierarchy. At the backend level, state is partitioned first by key-group and then by namespace before it reaches a compact sub-table. When the namespace is the common void namespace used by many keyed operators, the extra namespace map can be skipped and the backend addresses one table per key-group directly; otherwise each key-group owns a namespace-to-table map, and empty namespaces are removed eagerly after clears. This design reduces routing overhead on the hot path while keeping state isolation explicit enough to visualize per-namespace skew during the demonstration.

The current implementation already provides three workload families that the demonstration can reuse directly. The first is a WordCount-style stress test that isolates high-frequency state updates. The second uses NexMark queries to exercise window-heavy and join-heavy streaming behavior [4]. The third uses mixed-state application kernels from the existing benchmark set to show realistic combinations of ValueState and MapState pressure. Together, they connect backend mechanisms with end-to-end workload behavior without changing tools or leaving the same interface.

## 2.2 Prototype Status

The BriskState prototype is already integrated with Flink-compatible state management and benchmark tooling. The delivered artifacts include the Java control layer and JNI bridge, the native engine shared library, and standalone benchmark jobs for WordCount and NexMark. The workflow

therefore builds on real workloads and recorded measurements rather than on a synthetic front end alone.

Figure 2 anchors the paper with an end-to-end comparison. On both platforms, state-intensive NexMark queries show large execution-time gaps once the heap backend encounters GC pressure or memory-sensitive configurations, and the same traces can be replayed to analyze why particular queries stall or recover under a different backend.

BriskState also remains faster at operator level. Across the 13 retained Intel **flink-state-benchmark** operations, the largest gains appear on list iteration (+132%), map insertion (+110%), map lookup (+91%), value insertion (+87%), and map removal (+74%). Together with Figure 2, these results connect a cheaper state path with faster full queries.

The workflow is organized in four stages:

- (1) select a workload and switch between heap-based and cache-aware backends;
- (2) observe time-series changes in throughput, tail latency, and checkpoint cost;
- (3) inspect which key-groups and namespaces become hot and how occupancy evolves;
- (4) replay a previously collected run to compare different execution traces.

The workflow focuses on the causal path from data layout to locality, runtime symptoms, and externally visible metrics.

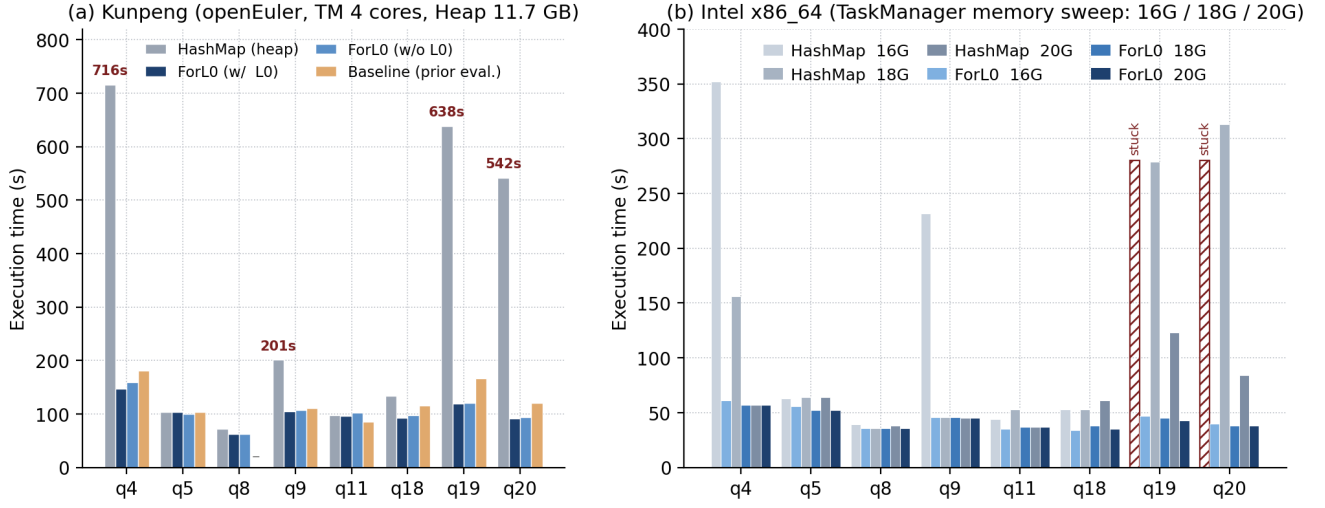
## 2.3 Implementation

The interface layer is designed to expose backend behavior without becoming a measurement artifact itself. Metrics are sampled at coarse intervals from the benchmark harness and Flink runtime, while the state view focuses on summaries such as namespace occupancy, key-group heat, and checkpoint-phase transitions.

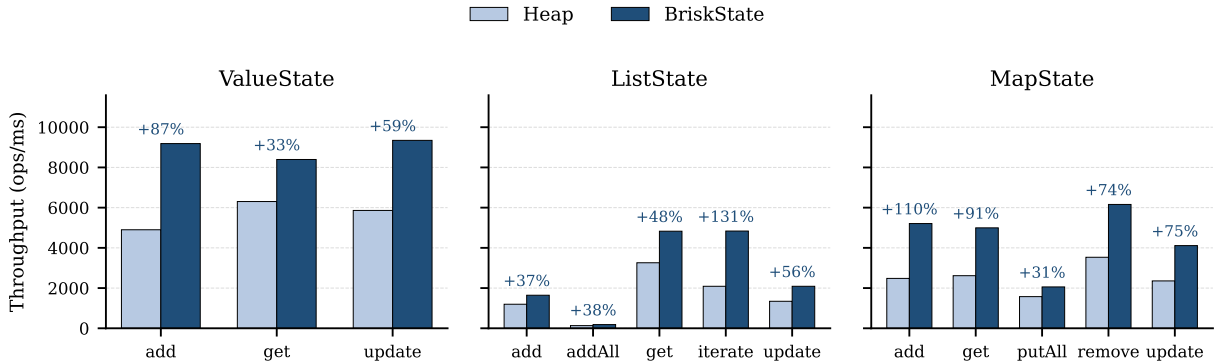
The backend implementation also exposes details that matter for interpretation. Within each sub-table, control metadata and payload arrays are organized to support compact probing and linear snapshot traversal, and the implementation includes specialized primitive-key paths that avoid boxing for common integer and long keys. Checkpointing follows Flink’s synchronous/asynchronous split: the synchronous phase captures table views using copy-on-write snapshot markers, while the asynchronous phase serializes those captured views in batches. This separation explains why checkpoint-related slowdowns sometimes appear as brief first-write penalties after a snapshot and sometimes as longer serialization tails, even when the user workload itself is unchanged.

The workflow supports both live execution and trace replay. Long-running queries and hardware-counter collection can be sensitive to machine load and deployment conditions, so the same interface can switch from streaming telemetry to recorded traces gathered from prior runs. This preserves comparability across runs while keeping the observed metrics and structural summaries in the same format.

BriskState supports both systems analysis and operational diagnosis. It exposes state-layout design, memory locality,



**Figure 2: Overall performance preview based on repository results.** The figure summarizes end-to-end behavior across representative NexMark queries on Kunpeng and Intel platforms, showing where BriskState avoids the large slowdowns and stuck configurations observed with the heap baseline.



**Figure 3: Operator-level Intel flink-state-benchmark results.** BriskState outperforms the heap backend across retained ValueState, ListState, and MapState operations.

and execution stability at a systems level, while also making it easier to distinguish operator logic, checkpoint cadence, backend choice, and state-structure imbalance in state-intensive Flink jobs.

### 3 Demonstration Scenario

The workflow begins with a side-by-side comparison mode. One workload is executed with two different backends, and the synchronized dashboard reports throughput, latency percentiles, checkpoint cadence, and backend-specific counters. This mode establishes that backend choice changes not only final job time but also runtime stability and responsiveness.

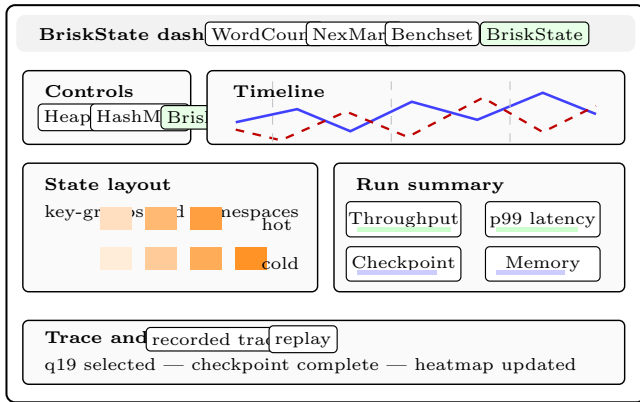
The second stage exposes state layout directly. The interface renders a hierarchical view from operator to key-group to namespace, then overlays heat and occupancy information to show where accesses concentrate. This view shows how a

smaller and more stable working set appears when namespaces are separated into compact sub-tables, and how that change affects the shape of the time-series plots.

Figure 4 shows the interface layout. The control panel starts or replays workloads, while the timeline aligns throughput, latency, and checkpoint events. A state inspector highlights key-group and namespace occupancy, and comparison cards summarize the current run pair. The same layout also supports recorded traces collected from prior runs, so live and replayed executions remain directly comparable.

### 4 Discussion and Conclusion

State backends are central to modern stream processing, but they remain difficult to reason about because their most important properties are physical rather than logical. BriskState combines backend structure, access locality, and runtime



**Figure 4: Interface sketch used in the paper. The layout combines workload selection, timeline comparison, state-layout inspection, run summaries, and trace replay in a single dashboard.**

**Table 1: Main UI panels and the telemetry each panel explains.**

| Panel            | Data source                         | Purpose                                                                                           |
|------------------|-------------------------------------|---------------------------------------------------------------------------------------------------|
| Control view     | Job launcher + saved traces         | How to switch workload, backend, and execution mode without leaving the same interface.           |
| Timeline         | Flink runtime and checkpoint events | When throughput drops align with latency spikes or checkpoint phases instead of operator logic.   |
| State inspector  | Demo collector summaries            | Which key-groups and namespaces are hot, sparse, or imbalanced during the run.                    |
| Comparison cards | Benchmark result summaries          | Which backend is faster or more stable for the chosen scenario and why the difference is visible. |

**Table 2: Scenarios covered in the demonstration.**

| Scenario               | Workload                | Focus                                                                                                                            |
|------------------------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| Pure state stress      | WordCount-style updates | Observe how backend choice changes update stability, throughput, and checkpoint cadence when the state path dominates execution. |
| Window-heavy streaming | NexMark queries         | Compare how latency spikes, hot namespaces, and run-to-run variability evolve under realistic stateful pipelines.                |
| Mixed state kernels    | Existing benchmark jobs | Inspect interactions among ValueState, MapState, and routing structure in more application-like operators.                       |

symptoms in a form that makes these designs easier to evaluate and reproduce. This combination of a systems prototype and a compact observation layer complements static plots by exposing parallel and memory-system behavior directly.

This paper presents BriskState together with a lightweight observation layer. By combining the delivered native state backend, existing benchmark artifacts, workload control, synchronized metrics, and trace replay, BriskState makes backend behavior directly observable. The result is a compact demonstration of how state layout affects stream-processing performance, stability, and debuggability.

## References

- [1] Abseil. 2018. Swiss Tables Design Notes. <https://abseil.io/about/design/swisstable>.
- [2] Tyler Akidau, Alex Balikov, Kaya Bekiroğlu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. 2013. MillWheel: Fault-Tolerant Stream Processing at Internet Scale. *Proceedings of the VLDB Endowment* 6, 11 (2013), 1033–1044.
- [3] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael J. Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. 2015. The Dataflow Model: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the VLDB Endowment* 8, 12 (2015), 1792–1803.
- [4] Apache Beam Community. 2024. NexMark Benchmark Suite. <https://beam.apache.org/documentation/sdks/java/testing/nexmark/>. Accessed 2026-04-24.
- [5] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, and Matei Zaharia. 2018. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark. In *Proceedings of the 2018 International Conference on Management of Data*. 601–613.
- [6] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. 2017. State Management in Apache Flink®: Consistent Stateful Distributed Stream Processing. *Proceedings of the VLDB Endowment* 10, 12 (2017), 1718–1729.
- [7] Paris Carbone, Asterios Katsifodimos, Stephan Ewen, Volker Markl, Seif Haridi, and Kostas Tzoumas. 2015. Apache Flink: Stream and Batch Processing in a Single Engine. *IEEE Data Engineering Bulletin* 38, 4 (2015), 28–38.
- [8] Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. 2018. FASTER: A Concurrent Key-Value Store with In-Place Updates. In *Proceedings of the 2018 International Conference on Management of Data*. 275–290.
- [9] Hyeontaek Lim Hoffman, Ryan Stutsman, Frank McSherry, and Malte Schwarzkopf. 2019. Megaphone: Latency-Conscious State Migration for Distributed Streaming Dataflows. In *Proceedings of the 2019 International Conference on Management of Data*. 435–450.
- [10] Derek G. Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martín Abadi. 2013. Naiad: A Timely Dataflow System. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. 439–455.
- [11] RocksDB Project. 2024. RocksDB. <https://rocksdb.org/>. Accessed 2026-04-24.